Contents

Figures and Tables 3
1. Project scope 4
1.1 Description of the application 4
1.2 Why is it useful? 4
1.3 Competitors 4
1.4 Target users 4
1.5 Breakdown of Perceived Costs 5
1.6 Monetisation of the product 6
1.7 Future enhancements 6
2. Software development 7
2.1 Requirements engineering 7
2.2 Software architecture 9
2.2.2 Process architecture – sequence diagram 11
2.3 Software design 13
2.4 Software development methodology 17
2.5 User acceptance testing recommendation 19
2.6 Documentation and libraries 19
2.6.1 Libraries concerning the product 19
3. Machine learning process 20
3.1 Gathering a dataset 20
3.1.1 Sourcing the data 20
3.1.2 Data quality 20
3.1.3 Loading the data into a database 20
3.1.4 Checking for bias 21
3.1.5 Cleaning my dataset 23
3.2 Choosing a model 24
3.3 Creating the model 24
3.3.1 Data preprocessing 25
3.3.2 Training the model 26

3.4 Implementing online learning 27
3.5 Testing. 29
3.5.1 Testing the initial model 29
3.5.2 Testing the model with real-world data 30
3.6 Refining the model 33
4. Application code 34
4.1 UI 34
4.2 Calculating the distance, elevation and weather 34
5. Retrospective 36
6. References ..... **Error! Bookmark not defined.**
7. AI prompts and links ..... **Error! Bookmark not defined.**

Figures and Tables

Figure 1 Logical view 10
Figure 2 Sequence diagram (next page) 11
Figure 3 Class Diagram 14
Figure 4 Data flow diagram 15
Figure 5 Map interactions UI 16
Figure 6 Kanban board 17
Figure 7 Example story in the Kanban board 17
Figure 8 Version control evidence 18
Figure 9 Testing plan 19
Figure 10 Requirements.txt 19
Figure 11 Kaggle data 20
Figure 12 CSV of merged data 21
Figure 13 Graphs code 21
Figure 14 Bar Chart comparing gender 22
Figure 15 Scatter graph and code 22
Figure 16 Database after dropping columns 23
Figure 17 Removing null data 23
Figure 18 High-level overview of the machine learning process 24
Figure 19 Labelling of data during preprocessing 25
Figure 20 Dataset after label Encoding 26
Figure 21 SGD model 26
Figure 22 Observations from training 27
Figure 23 Continuous Learning 28
Figure 24 Partial Training method 28
Figure 25 Metrics to evaluate performance 29
Figure 26 UI code 34
Figure 27 Elevation, distance and weather calculation 35

Table 1 Breakdown of costs 5
Table 2 Functional Requirements 7
Table 3 Non-functional requirements 8
Table 4 Evaluating different models (GeeksforGeeks, 2019) 24
Table 5 Test Cases evidence 30
Table 6 Retrospective 36

1. Project scope

1.1 Description of the application

The running predictor helps runners plot a route and displays an estimated time to complete it, based on age and gender specific averages for the distance drawn. As a reference, the application also shows the route's elevation, total distance, and weather.

Due to limited mobile development experience, I created a proof-of-concept (POC) on the web and aim to present my decisions from an investor's perspective.

1.2 Why is it useful?

In terms of cost, time, or effort, this application saves users time and effort when planning their workouts, improving productivity.

As a runner, planning my runs for the week is time-consuming, but this app helps me identify routes that fit my schedule. When preparing for races, I can evaluate my training by seeing how long it takes people in my age group to run the race route.

There’s also an element of discovery, as I can explore new routes in my area for a certain distance.

1.3 Competitors

Whilst Strava predicts race times and analyses your training, no existing application does what I am proposing for free, making it a unique selling point from other applications in this industry (Strava, 2024).

The site ‘Plot a Route’ (Plotaroute.com, 2019) allow users to plan a route but relies on a manual user guide to estimate the time.

1.4 Target users

The primary audience is runners, but defining its size isn't feasible because it spans from professional athletes to casual runners. Secondary users include sports coaches and running clubs that support athletes in their training.

1.5 Breakdown of Perceived Costs

The costs are broken down into annual expenses. For the POC, the application is deployed on the StreamLit cloud, however, for enterprise use, I recommend using AWS with AWS RDS as the cloud provider.

Table 1 Breakdown of costs

Tool/ Persona	**Cost Type**	**Explanation**	**Predicted cost**
AWS EC2	Deployment cost	Deploying to the cloud with on-demand scalability provides the app with more computing power during race season than during the holiday period.	$616.68 (AWS, 2025a)
Engineer	Developer hours	It took about 30 hours to build this application, and the average developer's pay is $30 per hour. Development hours per week would average 40 hours.	$62400
Selenium	Automated testing	No cost, but learning time should be factored in.	$0 (ZenRows, 2023)
GitHub	Task management	Version control and project planning for businesses is $21 per month.	$252 (Github, 2025)
Lucid chart	Diagrams and design	Documentation of ongoing enhancements of the application	$120 (LucidChart, 2015)
UI designer	UI design	The average developer's pay is $25 per hour. Development hours per week would average 8 hours.	$50400
Wix	Personalised domain name	Migrating from localhost to a personalised URL enhances credibility with a .com domain name.	$13.35 (Shwake, 2023)
Grafana	Analytics and metrics	Enterprise-grade monitoring, visualisations, and metrics tool with custom support.	$25,000 (Grafana, n.d.)
AWS RDS and Postgres	Database with database backups	To support AWS cloud deployments, it is necessary to use their PostgreSQL offering with data backups as a recovery strategy.	$ 99,781.68 (AWS, 2025b)
AWS Macie	Security	Handling location, age, and gender data means that data protection and monitoring are a priority.	$1194 (Calculator.aws, 2025)

The *total* annual cost to develop an enterprise-level MVP for this application is $239,777.71.

1.6 Monetisation of the product

This application uses a freemium model to allow users to plot their routes and receive generalised predictions from a non-personalised dataset.

To receive personalisation and connect to other applications, users need to subscribe to the app for around £4 a month. To save routes to an account, there's an additional charge, bringing the fee to £6 per month.

1.7 Future enhancements

Additional features for development after the POC:

- Login features to allow users to save their routes.
- Integration with third-party apps.
- Based on the user's training goals, the application will recommend distances and, hence, appropriate routes.

2. Software development

2.1 Requirements engineering

2.1.1 Functional requirements – User Stories

Table 1 shows the user stories required, where the ‘As a ' represents an actor, ‘I want’ specifies the feature, and ‘So that’ describes the reasoning.

As this project is a POC and to maintain high quality, I have a small set of functional requirements.

Table 2 Functional Requirements

ID	As a	I want to...	So that I can...	Priority
1	Runner	Interact with a map	I can determine a route for my run.	High
2		Plot a route	The application can estimate the time to complete the run based on the route's distance and elevation.	
3		Receive a predicted time for my plotted route	I can plan out my training sessions for a run.	
4		Enter my age group.	I can understand what time is expected for my age group.	
5		Enter my gender	I can find out what the average pace for people running this distance is.	
6		Know the elevation of my route.	I can determine how challenging the route will be.	Medium

2.1.2 Non-Functional requirements

Being a machine-learning focused POC, requirements one to six, the non-functional requirements in Table 3, are the qualities I wanted from my machine learning model.

Table 3 Non-functional requirements

ID	**Category of requirement**	**Type**	**Description**	Priority
1	Process	Retainability	The Machine learning model should adapt to new data and continuously refine its findings.	High
2		Accuracy	The model should give a reasonable prediction of how long it will take a runner to complete a route.	High
3		Simplicity	Due to time constraints, the project's scope is locked down by a fixed deadline.	Low
4	Structural	Data integrity	The data used to train the models should maintain referential integrity, remain confidential, and be available.	High
5		Modularity	Each domain of the system should loosely couple with the other components of the application.	High
6		Extensibility	New features should be easy to add beyond the POC.	Medium
7	Operational	Useability	The User Interface (UI) should be intuitive to navigate with an accessible layout.	High

2.2 Software architecture

2.2.1 Architecture overview and technology stack

Investors will favour architectures that can adapt to changing demands. As shown in Figure 1, a three-tiered architecture supports this by minimising coupling between the business, presentation, and database layers, allowing features to be enhanced and scaled independently (Chiaramonte, 2024).

From an investor's perspective, this architecture allows developers to work on parts of the product in parallel without dependencies, improving reliability and security, as each layer can be worked on to its specific needs.

For the database layer, PostgreSQL (PostgreSQL Group, 2025) was chosen as it's open source, eliminating licensing costs, and making it suitable for vertical scaling as the number of users increases.

For the UI in the presentation layer, I chose the Streamlit Folium Python package (Streamlit Folium, n.d) for its geospatial capabilities and interactions. It provides a modern ReactJS-like interface, without needing JavaScript or HTML. This library is ideal for enterprise use due to its excellent documentation, ease of use, and appealing interface. Investors don't need to hire JavaScript-skilled developers, they save costs by hiring engineers with only Python experience.

The business logic layer contains a set of classes that fulfil the functional requirements, including machine learning activities. In this layer, only one class interacts with the database, ensuring that all queries are executed through a single-entry point, promoting maintainability and security. Similarly, the presentation layer manages the user interactions in a single file, and user input is passed as parameters to the relevant classes.

The solution is written in Python because it's a flexible language for database interactions, supports a UI with Streamlit Folium, and has built-in machine learning libraries.

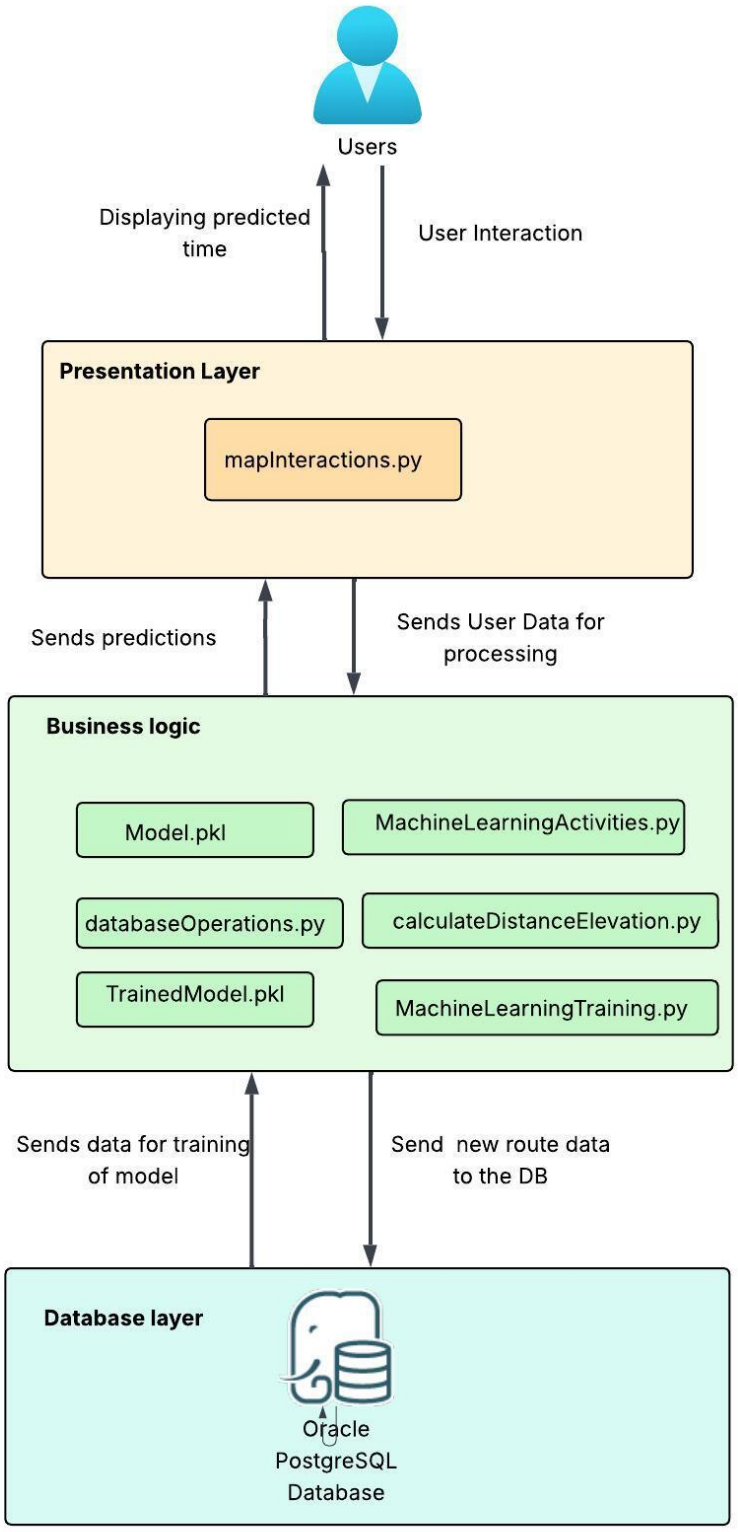
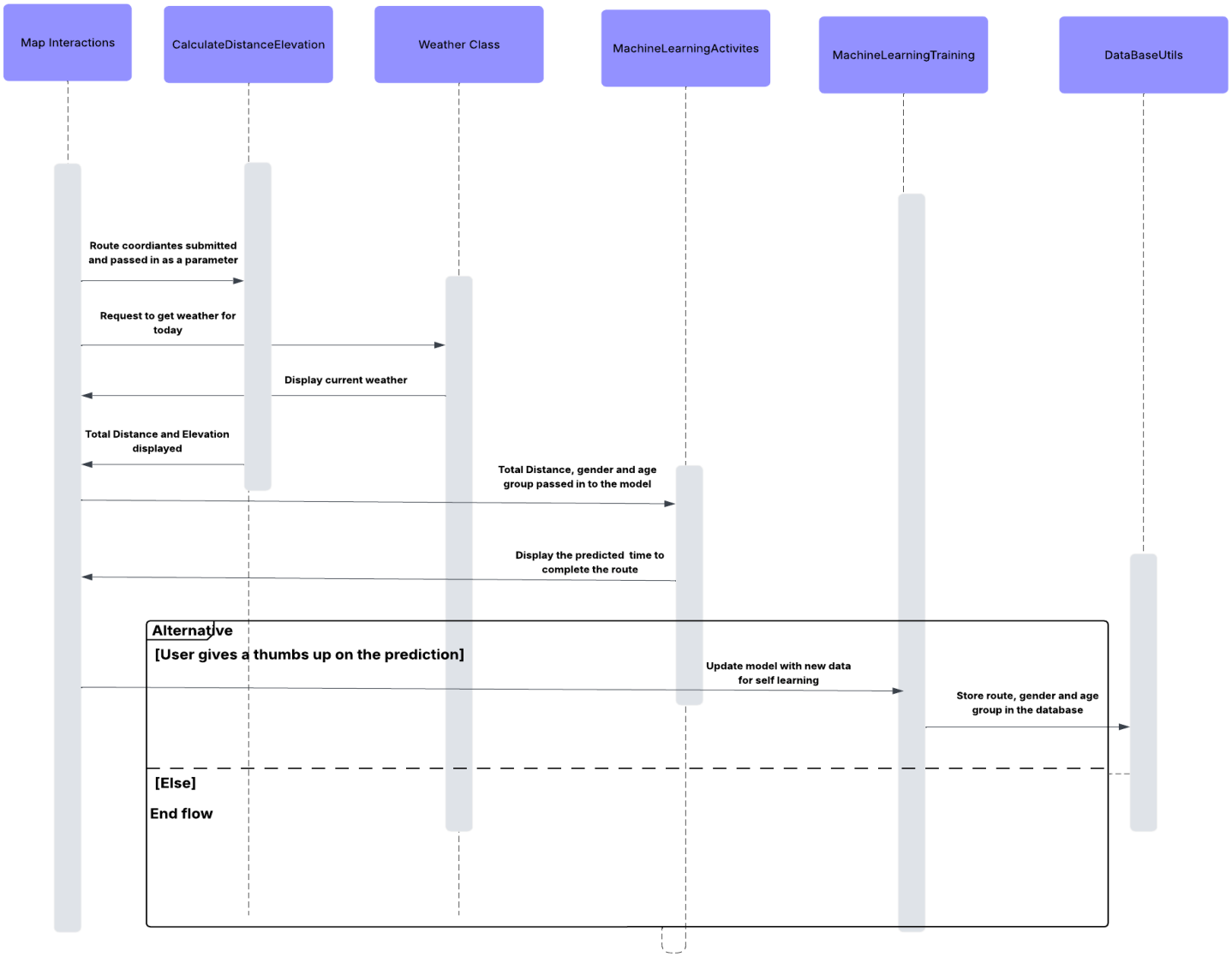


Figure 1 Logical view

2.2.2 Process architecture – sequence diagram

The process architecture view in Figure 2 shows the application's runtime activities. Notably, once the predicted time and route are displayed, the user sees a feedback button. If they press ‘thumbs up’, the retrain feature will run, adding the user’s route, age, and gender to the database to update the model.

Figure 2 Sequence diagram (next page)


Map Interactions
CalculateDistanceElevation
Weather Class
MachineLearningActivites
MachineLearningTraining
DataBaseUtils
Route coordiantes submitted and passed in as a parameter
Request to get weather for today
Display current weather
Total Distance and Elevation displayed
Total Distance, gender and age group passed in to the model
Display the predicted time to complete the route
Alternative
[User gives a thumbs up on the prediction]
Update model with new data for self learning
Store route, gender and age group in the database
[Else]
End flow

2.3 Software design

2.3.1 Structural architecture

The class diagram in Figure 3 breaks down the application into logical components that promote single responsibility and dependency inversion, helping investors understand how object-oriented programming contributed to the system design.

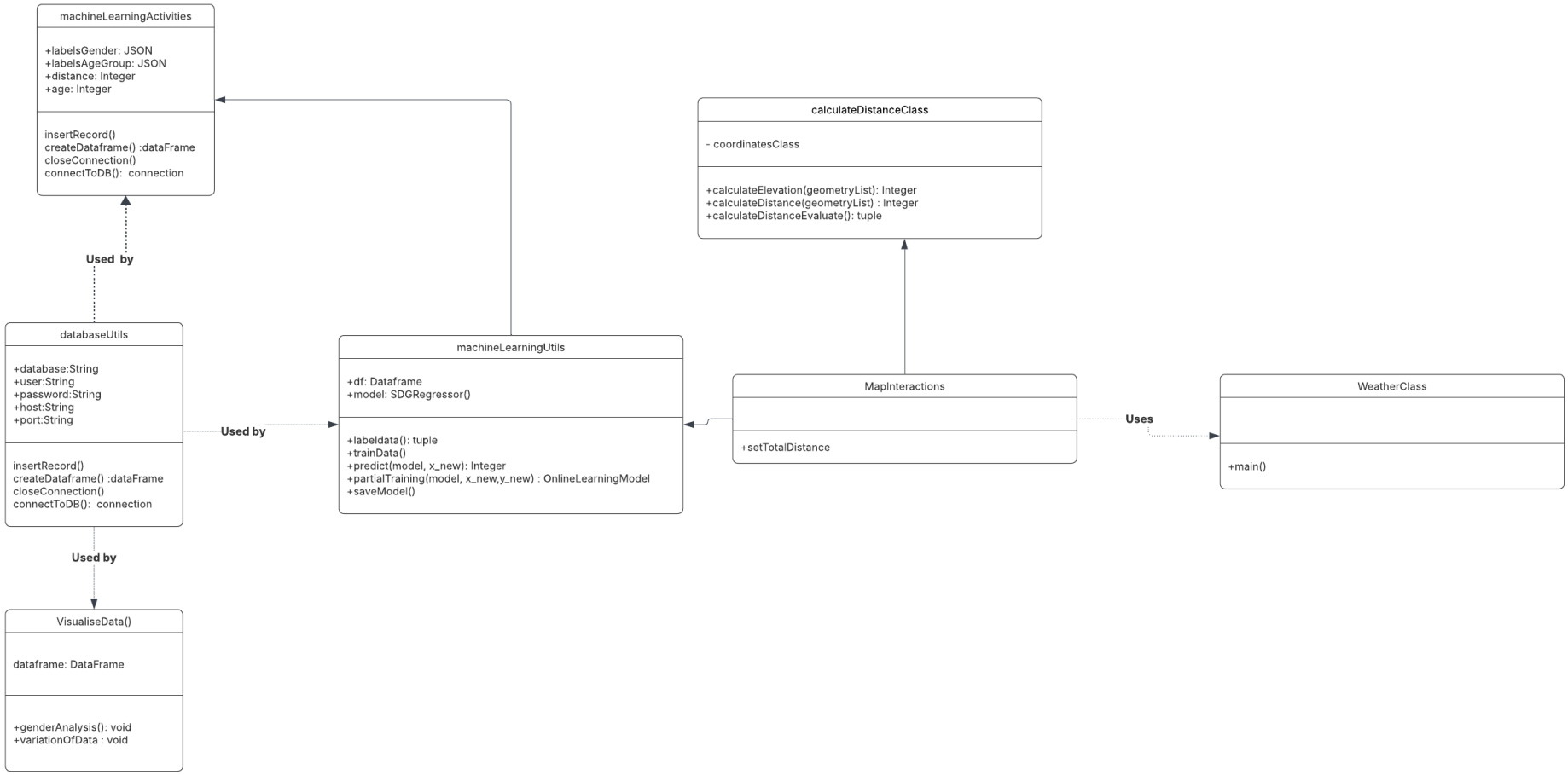


Figure 3 Class Diagram

2.3.2 Data flow diagram

Figure 4 shows the interaction between data points across the application, enabling developers to identify the purpose of each domain.

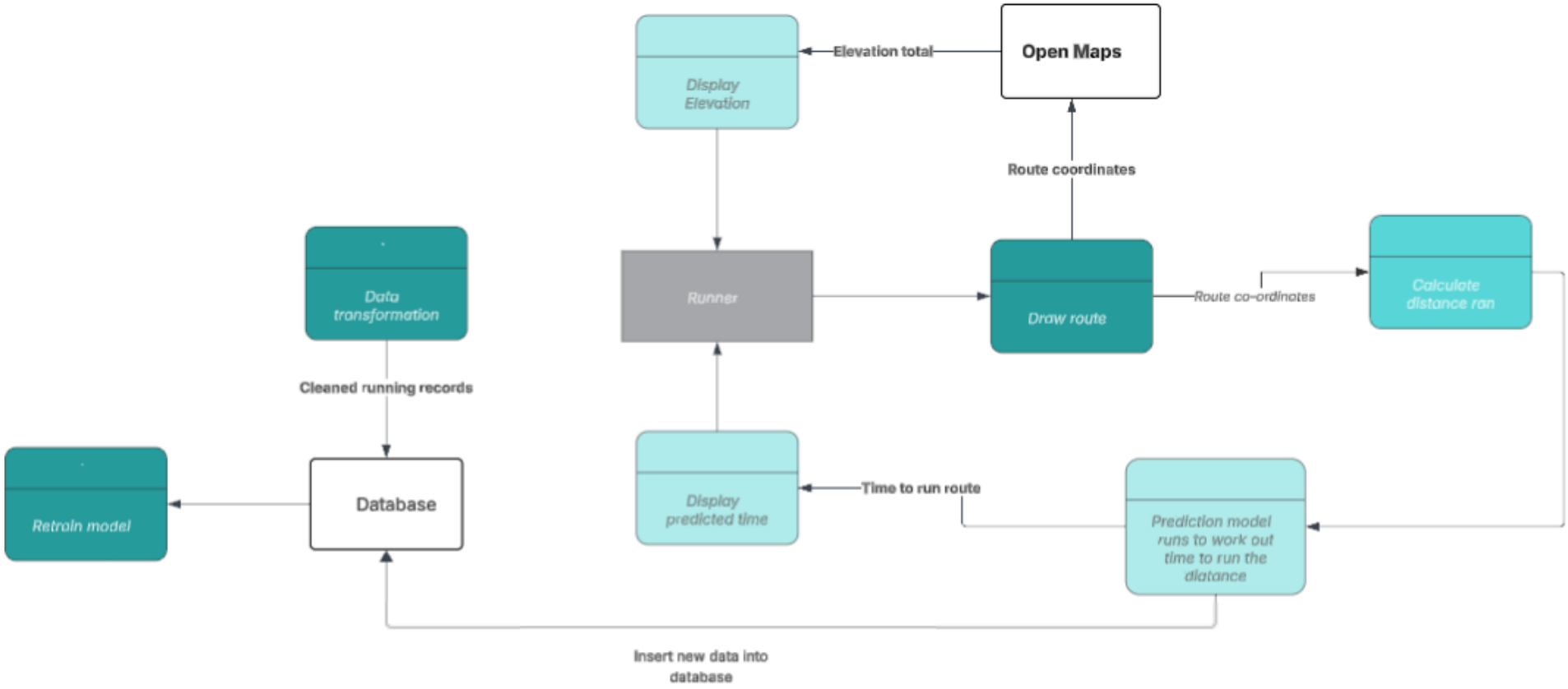


Figure 4 Data flow diagram

2.3.3 Wireframes

There is a single webpage for the UI because my POC serves a specific function. Figure 5 displays the interactive map and accompanying route information.

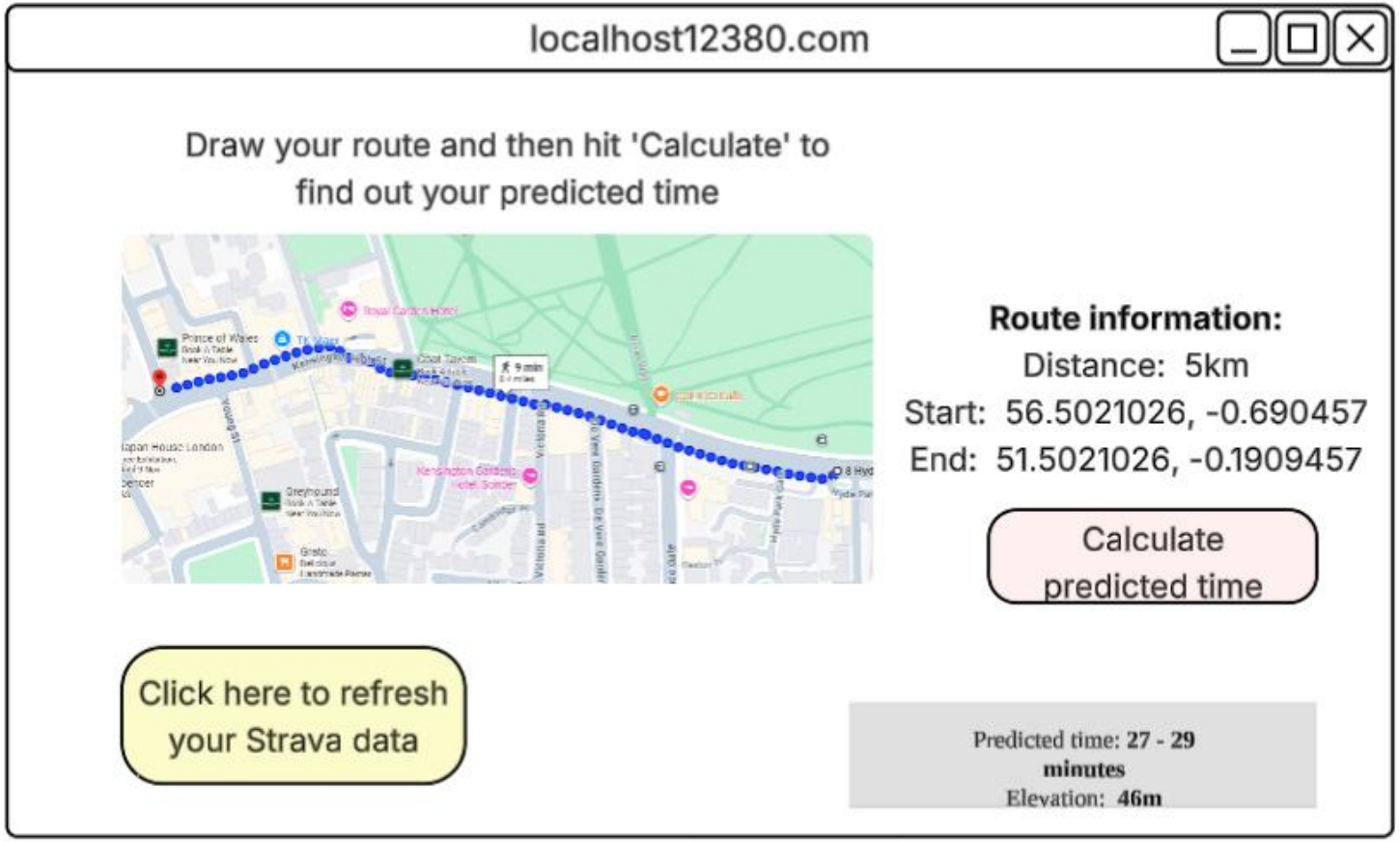


Figure 5 Map interactions UI

2.4 Software development methodology

2.4.1 Software development during the POC.

As a solo developer with varied availability, I used Kanban over Scrum because set sprints in Scrum weren't practical. Kanban allowed me to consistently add value to my project with a reliable way to track tasks, as shown in Figure 6 on GitHub. Kanban was more suitable than waterfall, given the iterative design and growing understanding of machine learning.

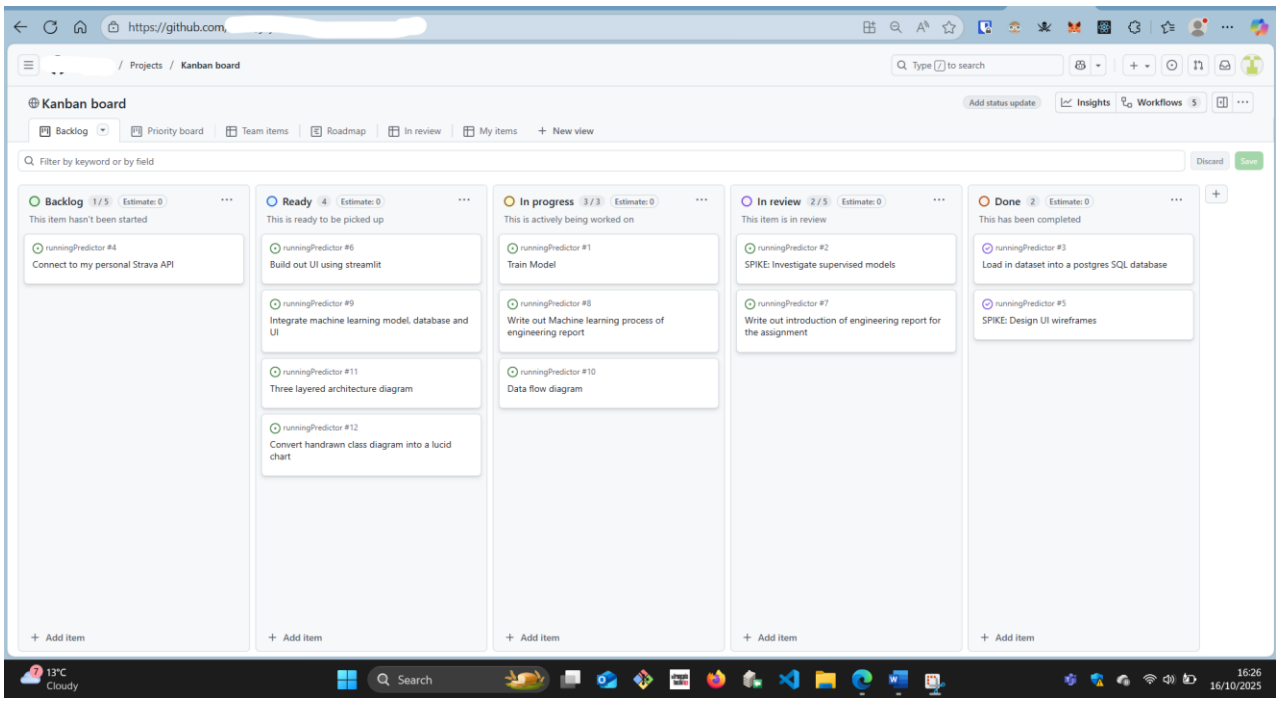


Figure 6 Kanban board

Following the 'source, context, ask' framework for writing my stories to provide clear outcomes for each ticket. Figure 7 shows an example story.

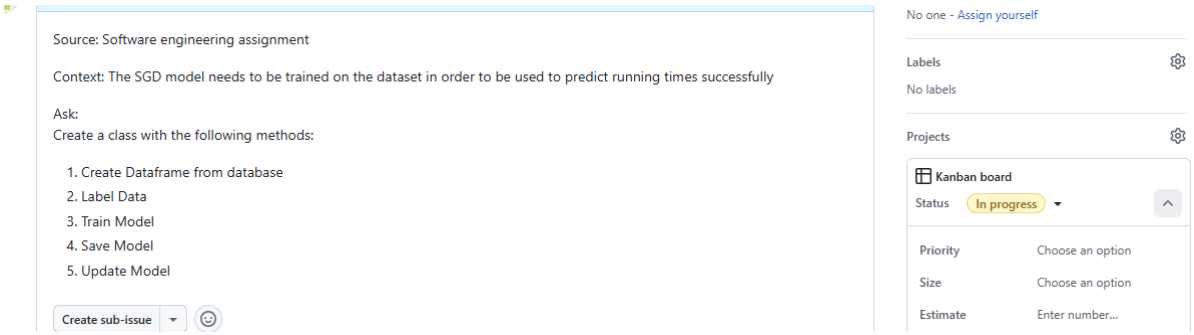


Figure 7 Example story in the Kanban board

2.4.2 Software development methodology beyond the POC

I recommend that customers continue using Kanban beyond the POC, as traditional Scrum requires time estimates. Due to the unreliability of machine learning systems and ongoing design efforts, Kanban enables tracking work while allowing flexibility in timelines.

2.4.3 Version control for clients

I created a GitHub repository, shown in Figure 8, to store my code, and the clients will expand it for enterprise use by subscribing to GitHub's business plan.

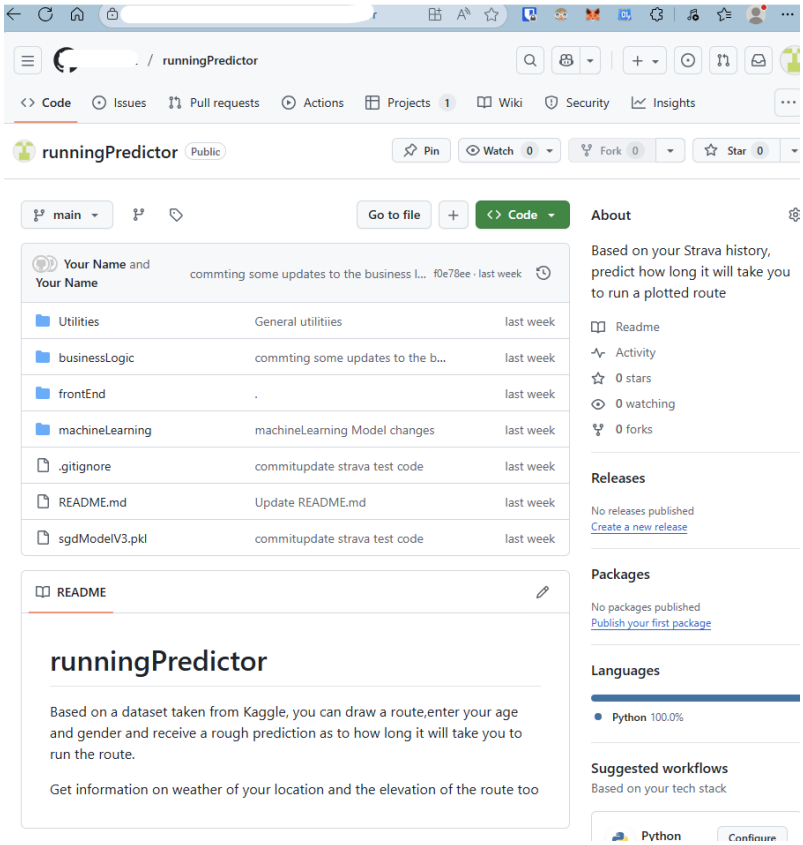


Figure 8 Version control evidence

2.5 User acceptance testing recommendation

For both investors and me, the testing plan in Figure 9 should be used to gather feedback for the application from a usability and stakeholder satisfaction perspective.

```
Testing plan:

1. Submit the route without anything on the map -  validate an error returns back
2. Draw a route, enter the age group and gender. Press submit and validate that the elevation ,weather, and route
gets populated on the web page
3. Validate that you can interact with the thumbs up and thumbs down buttons that appear once the
route information has been populated
4. Validate that you can zoom out and in with the map and can break up your route into segments
5. Repeat for a range of age groups and genders so that we can prove that the time taken to create the route
varies accross the different categories
```

Figure 9 Testing plan

2.6 Documentation and libraries

For POCs, documenting the libraries used, software architecture diagrams (section 2.2), and high-level testing plans is essential to helping clients understand what tools were used for the proposed idea.

2.6.1 Libraries concerning the product

The requirements.txt in Figure 10 is necessary for the investors from a security perspective because my application uses personal data such as age and gender, and could reveal users' locations. Listing the libraries allows investors to assess any risks associated with the application's dependencies.

requirements.txt

```
contourpy==1.3.3
elevation==1.1.3
folium==0.20.0
geopy==2.4.1
matplotlib==3.10.7
mdurl==0.1.2
meteostat==1.7.6
numba==0.62.1
numpy==2.2.6
oauthlib==3.3.1
opencv-python==4.12.0.88
pandas==2.3.3
pillow==11.3.0
pyhigh==0.0.6
python-weather==2.1.0
scikit-learn==1.7.2
scipy==1.16.2
six==1.17.0
smmap==5.0.2
SQLAlchemy==2.0.44
streamlit==1.50.0
streamlit-feedback==0.1.4
streamlit-folium==0.25.3
```

Figure 10 Requirements.txt

3. Machine learning process

3.1 Gathering a dataset

3.1.1 Sourcing the data

The primary data needed were running activities that contained total distance and time. Initially, I tried using my own Strava data, but a small dataset of 400 points risked leading to overfitting. Therefore, I used the long-distance runner dataset from Kaggle (Figure 12), made from 10,703,690 records (beridzeg45, 2025).

Although the model is not truly personalised to the athlete, it would be more precise because the dataset included age and gender information for the athletes.

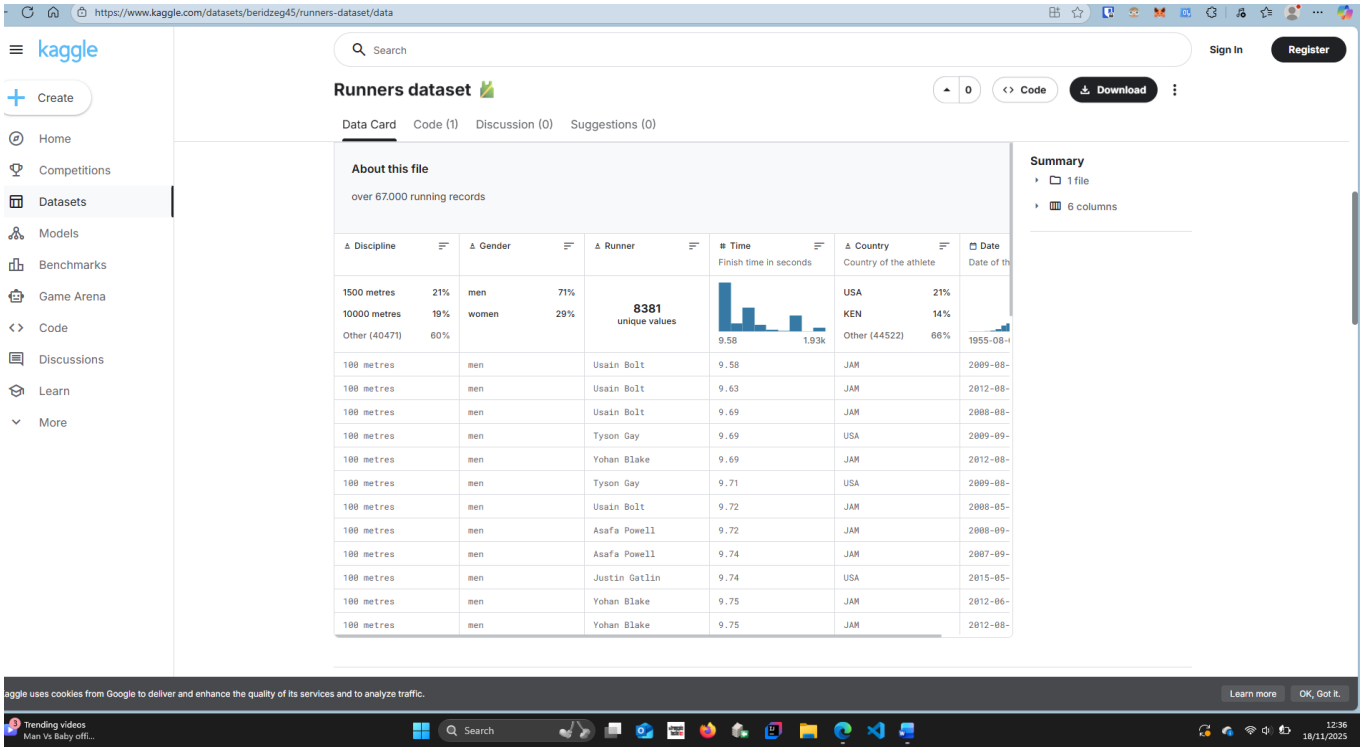


Figure 11 Kaggle data

3.1.2 Data quality

In the dataset, all data were prelabelled with gender, age group, distance in kilometres, and time taken in minutes, so this didn't need to be done manually.

3.1.3 Loading the data into a database

The Kaggle dataset was split into five files, so I merged the records into a single CSV file and bulk-loaded them into a database. Using a database makes the data more manageable while allowing new records to be added for continuous learning.

	A	B	C	D	E	F	G	H	I	J	K	L
1	activityid	datetime	athlete	distance	duration	gender	age_group	country	major			
2	0	01/01/2019	0	0	0	F	18 - 34	United States	CHICAGO 2019			
3	1	01/01/2019	1	5.27	30.2	M	35 - 54	Germany	BERLIN 2016			
4	2	01/01/2019	2	0	0	M	35 - 54	United Kingdc	LONDON 2018,LONDON 2019			
5	3	01/01/2019	3	10.5	43.95	M	18 - 34	United Kingdc	LONDON 2017			
6	4	01/01/2019	4	9.66	48.65	M	35 - 54	United States	BOSTON 2017			
7	5	01/01/2019	5	10.38	50.1333333	F	35 - 54	United States	BOSTON 2015			
8	6	01/01/2019	6	10.11	53.1833333	M	55 +	United States	BOSTON 2017			
9	7	01/01/2019	7	0	0	M	35 - 54	Australia	CHICAGO 2012			

Figure 12 CSV of merged data

3.1.4 Checking for bias

To understand how my data was categorised and varied, I ran a series of Matplotlib graphs.

Bar chart comparing the gender ratio

My dataset was heavily biased towards male runners, reflecting societal bias in the athletic world and likely reducing the prediction accuracy for females. Figure 14 shows the code to create this graph, and Figure 15 shows the output (GeeksforGeeks, 2020a).

Figure 13 Graphs code

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import joblib
import matplotlib
import matplotlib.pyplot as plt

class visualiseData():
    def __init__(self):
        self.df=databaseUtils().convertToDf()

    def gender_analysis(self):
    # Plot bar chart using Pandas
        self.df['gender'].value_counts().plot(kind='bar', color='skyblue', title='Count of gender')

        plt.title("Dataset by gender")
        plt.xlabel("gender")
        plt.ylabel("Count of gender")
        plt.show()
```

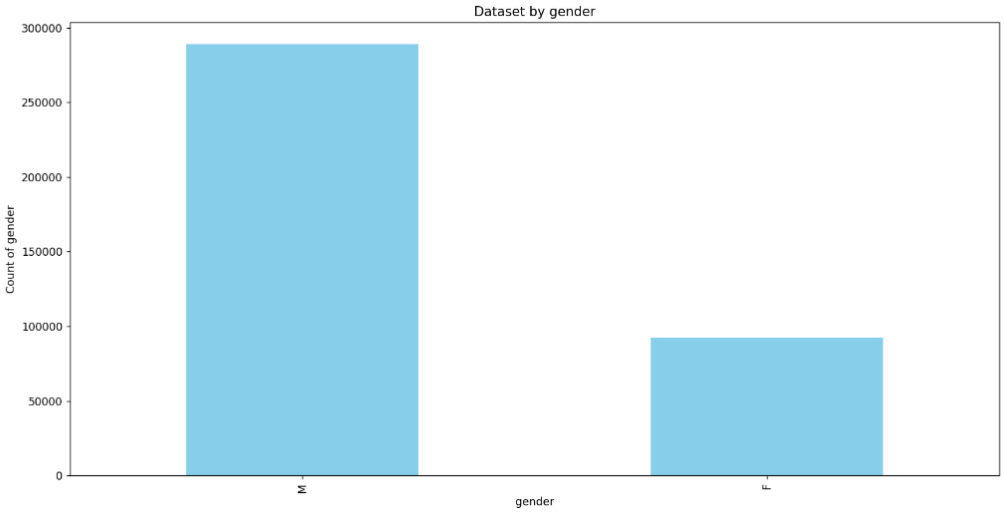
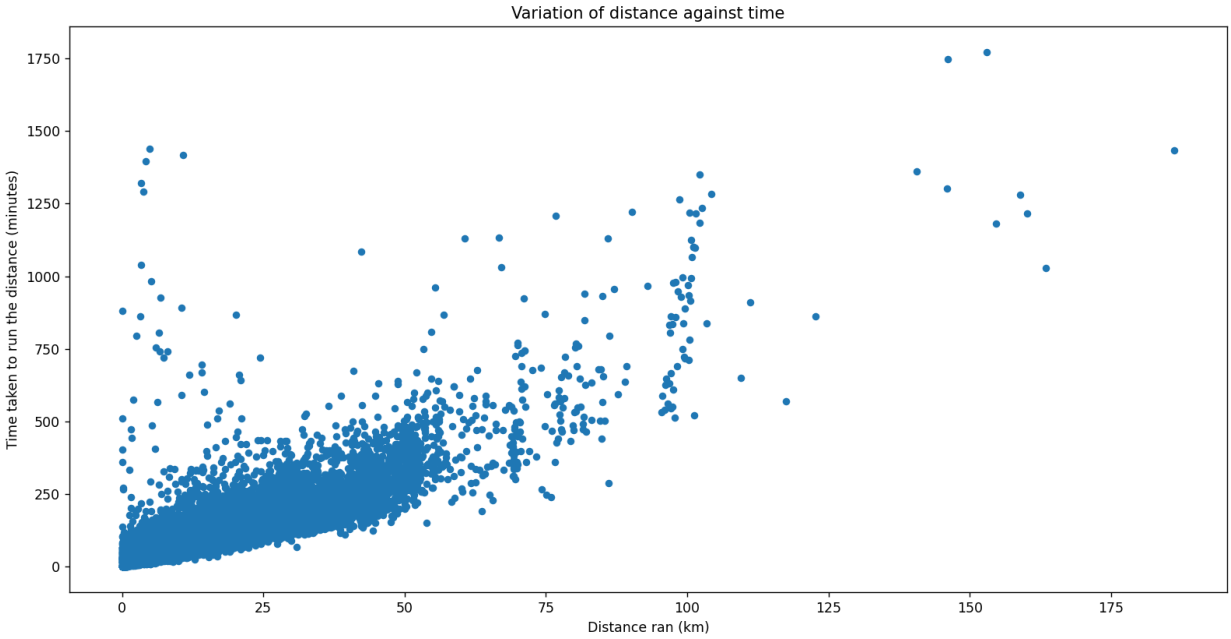


Figure 14 Bar Chart comparing gender

Scatter graph showing the range of distances against the times

Figure 16 shows that most distances in my dataset were between 0 and 50km, with the times clustered together. However, the few outliers contributed to the model's variance, increasing the risk of inaccurate predictions (Team, 2022).




```python
def variationOfdistance(self):

    # Creating a scatter plot
    self.df.plot.scatter(x='distance' , y='duration')
    plt.title("Variation of distance against time")
    plt.xlabel("Distance ran (km)")
    plt.ylabel("Time taken to run the distance (minutes)")
    plt.show()
```

Figure 15 Scatter graph and code

3.1.5 Cleaning my dataset

I first removed unwanted columns, such as ‘Major’, ‘Country’, ‘Date’, and ‘Athlete’, with the resulting columns shown in Figure 17.

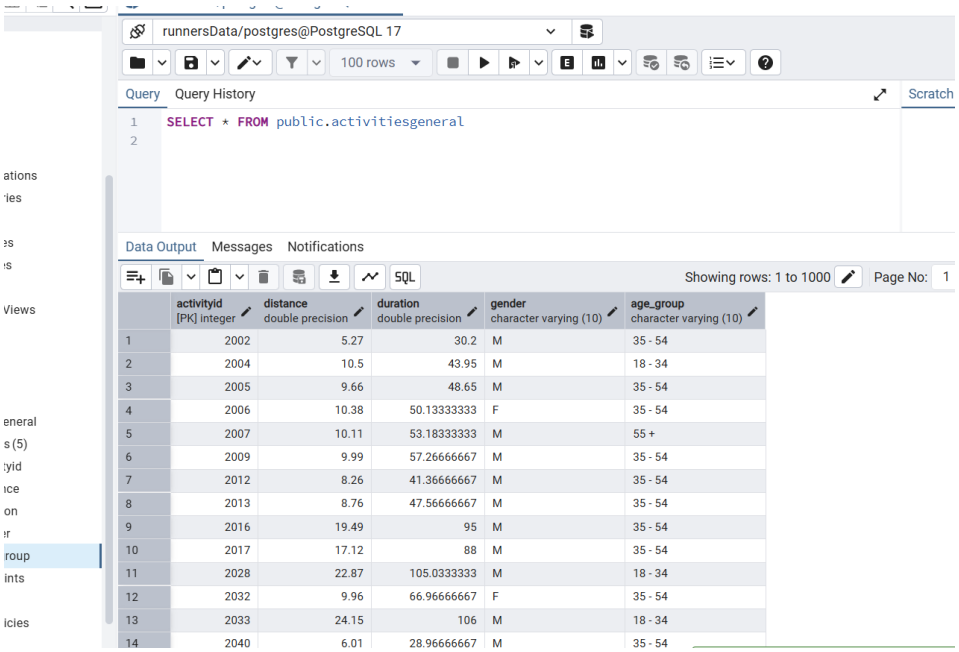


Figure 16 Database after dropping columns

Figure 18 shows the SQL query returned over 600,000 records with a ‘0’ duration, indicating no activity was recorded. Therefore, the second part of data cleaning involved deleting all these rows.

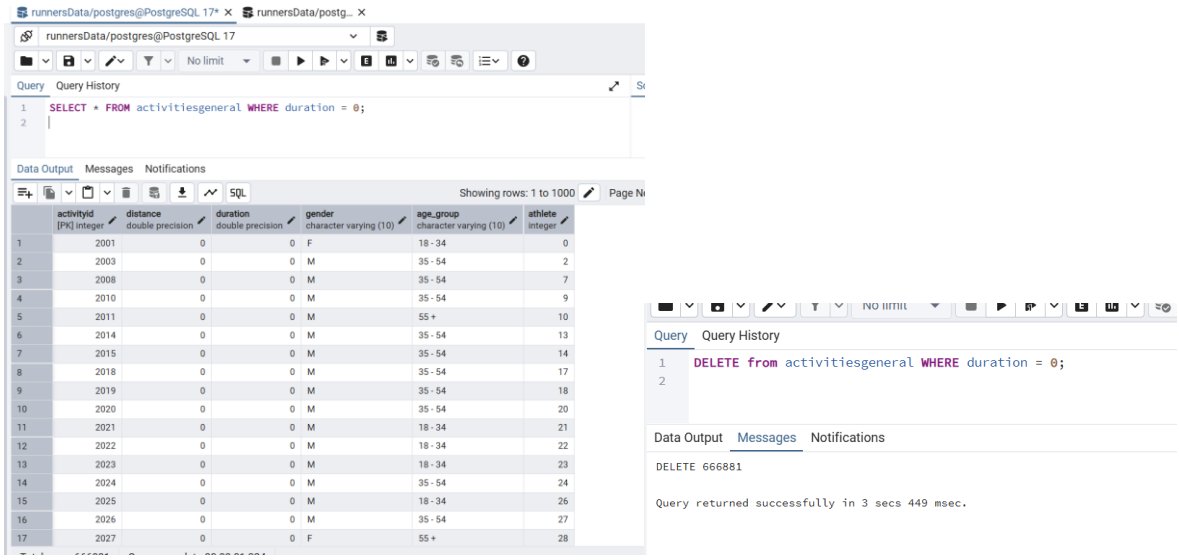


Figure 17 Removing null data

3.2 Choosing a model

A supervised approach was necessary because I had pre-labelled data, and regression was used to learn patterns and make predictions. Table 4 shows the models that were **not** suitable.

Table 4 Evaluating different models (GeeksforGeeks, 2019)

Model	**Usage**	**Why was the model not chosen?**
Linear regression	Showing a relationship between independent and dependent variables	Doesn't handle outliers well and assumes the relationship is linear, which is not true for fitness data.
Random forest tree	Multiple decision trees, and each tree is trained on a random subset of the data.	Doesn't support continuous learning of data
Decision tree	Data can be trained on a single tree and doesn't require intensive amounts of energy.	Not suitable for larger data sets and variables like age and gender

I chose a *Stochastic Gradient Descent* (SGD) regressor, which trains on small batches of data over multiple iterations, aiming to reduce the error between the actual and predicted values (the loss function). This feature enables the SGD model to perform live retraining via online learning and handle large volumes of data (Otten, 2025).

I created a new model rather than using transfer learning to have complete control over the parameters. A simple model was chosen over a deep learning model, as the patterns for predicting time were not complex and didn't need excessive computation.

3.3 Creating the model

Figure 11 presents the process I followed for training my machine learning model.

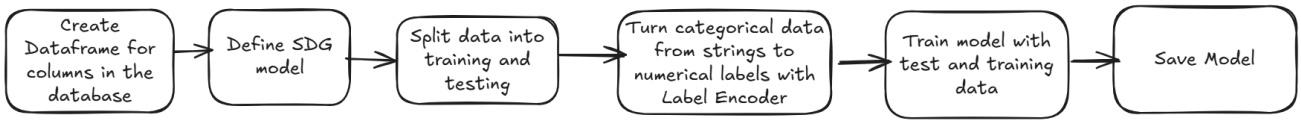


Figure 18 High-level overview of the machine learning process

3.3.1 Data preprocessing

The Gender and Age Group columns were strings, so as shown in Figure 19, I used LabelEncoder, a method in 'sklearn preprocessing' that automatically converts categories into numerical values the model can understand. (Bobbitt, 2022).

In the TrainModel class, the labelData method instantiates two objects for each database column that needs categorising. For each column, the fit_transform method learns the categories, assigns each category a numerical value, and replaces the column's data with the new values.

```
class trainModel():
    #open connection
    def __init__(self):
        self.df=databaseUtils().convertToDf()
        self.model = SGDRegressor(max_iter=1000, alpha=0.0001, learning_rate='invscaling', random_state=42)


    def labelData(self):
        """
        Label the gender and age group columns as part of training the model
        """
        le_gender = LabelEncoder()
        le_age_group = LabelEncoder()
        gender_map = le_gender.fit_transform(self.df['gender'])
        #print(le_gender.classes_)
        #print(self.df['gender'])
        age_group_map = le_age_group.fit_transform(self.df['age_group'])
        gender_labels = dict(zip(self.df['gender'], gender_map))
        # get the mapping between the original labels and encoded labels
        age_group_labels = dict(zip(self.df['age_group'], age_group_map))

        self.df['age_group'] = age_group_map
        self.df['gender'] = gender_map
        #print(le_age_group.classes_)
        return gender_labels,age_group_labels

```

Figure 19 Labelling of data during preprocessing

Figure 20 shows the results of Label Encoding.

distance	gender	age_group
8.870	1	1
12.060	1	1
8.119	1	1
7.010	1	0
2.009	1	1
...	...	...
13.760	1	1
13.140	0	0
14.730	1	0
5.810	1	1
6.620	1	0

Figure 20 Dataset after label Encoding

3.3.2 Training the model

Figure 21 shows how I used the Scikit-learn library to build and train the SGD model. (Scikit-learn, n.d.).

```
You, 3 days ago | 1 author (You)
class trainModel():
    #open connection
    def __init__(self):
        self.df=databaseUtils().convertToDf()
        self.model = SGDRegressor(max_iter=2000, alpha=0.0001, learning_rate='invscaling', random_state=42)
```

Figure 21 SGD model

The initial hyperparameters (Scikit Learn, n.d) I used were:

- Max_inter: Maximum number of training iterations the model experiences.
- Alpha: Method to reduce overfitting.
- Learning_rate: Control learning speed as the dataset decreases
- Random State: How many times the data is split.

Setting ‘verbose=1’ enabled monitoring of the loss function, which decreased overall, proving that prediction errors decreased (see Figure 22). An interesting observation was that the bias fluctuated widely across epochs, suggesting that the dataset's heavy male bias influenced the predictions. Unfortunately, it wasn’t possible to visualise this data easily because the parameter prints the results without saving them.

```
You, 24 seconds ago | 1 author (You)
class trainModel():
    #open connection
    def __init__(self):
        self.df=databaseUtils().convertToDf()
        self.model = SGDRegressor(max_iter=1000, alpha=0.0001, learning_rate='invscaling', random_state=42, verbose=1)
```

PROBLEMS OUTPUT TERMINAL PORTS GITLENS Python

```
-- Epoch 1
Norm: 10.32, NNZs: 3, Bias: 0.206017, T: 305356, Avg. loss: 294.204470
Total training time: 0.03 seconds.
-- Epoch 2
Norm: 9.43, NNZs: 3, Bias: 1.193140, T: 610712, Avg. loss: 235.330361
Total training time: 0.07 seconds.
-- Epoch 3
Norm: 10.14, NNZs: 3, Bias: 1.244446, T: 916068, Avg. loss: 226.732397
Total training time: 0.10 seconds.
-- Epoch 4
Norm: 9.77, NNZs: 3, Bias: 1.270213, T: 1221424, Avg. loss: 219.204787
Total training time: 0.13 seconds.
-- Epoch 5
Norm: 10.11, NNZs: 3, Bias: 1.414262, T: 1526780, Avg. loss: 214.939327
Total training time: 0.17 seconds.
-- Epoch 6
Norm: 10.94, NNZs: 3, Bias: 0.287304, T: 1832136, Avg. loss: 212.047514
Total training time: 0.21 seconds.
-- Epoch 7
Norm: 9.89, NNZs: 3, Bias: 2.050612, T: 2137492, Avg. loss: 211.594825
Total training time: 0.24 seconds.
-- Epoch 8
Norm: 9.81, NNZs: 3, Bias: 2.095458, T: 2442848, Avg. loss: 209.623997
Total training time: 0.28 seconds.
-- Epoch 9
Norm: 10.89, NNZs: 3, Bias: 0.403486, T: 2748204, Avg. loss: 207.218679
Total training time: 0.32 seconds.
-- Epoch 10
Norm: 10.71, NNZs: 3, Bias: 0.445535, T: 3053560, Avg. loss: 206.221851
Total training time: 0.36 seconds.
-- Epoch 11
Norm: 10.02, NNZs: 3, Bias: 1.875461, T: 3358916, Avg. loss: 205.530102
Total training time: 0.40 seconds.
-- Epoch 12
Norm: 10.02, NNZs: 3, Bias: 2.059725, T: 3664272, Avg. loss: 203.260810
Total training time: 0.44 seconds.
```

Figure 22 Observations from training

3.4 Implementing online learning

When the user presses the thumbs-up button for plotted routes, the model processes the new data in real time and fine-tunes its predictions. Figure 23 shows the class initialised with data from the UI, the pretrained model, and labels that numerically categorise gender and age groups.

The decorator method first calculates the predicted time using the trained model. The model's result is then inserted into my database along with the distance, age group, and gender.

Finally, in line 35, the model retrains itself by calling the ‘Partial Training’ method, shown in Figure 24.

```python
class machineLearningUtils():
    def __init__(self,distance,age,gender):
        self.labelsGender = {'M': np.int64(1), 'F': np.int64(0)}
        self.labelsAgeGroup= {'35 - 54': np.int64(1), '18 - 34': np.int64(0), '55 +': np.int64(2)}
        self.distance = int(distance)
        self.age = age
        self.gender = gender
        self.model = joblib.load(open(str('sgdModelV2.pkl'), 'rb'))

    def createNpArray(self):
        ageLabel = self.labelsAgeGroup[self.age]
        genderLabel = self.labelsGender[self.gender]
        data =[[self.distance,genderLabel,ageLabel]]
        np_array = pd.DataFrame(data,columns=['distance','gender','age_group'])
        return np_array

    def decorator(function):
        def wrapper(self):
            predictedTime =function(self)
            query = "INSERT INTO activitiesGeneral (distance, duration,gender, age_group) VALUES (%s,%s, %s, %s)"
            values = (self.distance, np.array(predictedTime).item(), self.gender, self.age)
            databaseUtils().writeToDb(query,values)
            trainModel().partialTraining(self.model, self.createNpArray(), predictedTime)
            joblib.dump(self.model, open('sgdModelV2.pkl', 'wb'))
            return predictedTime
        return wrapper

    @decorator
    def predictModel(self):
        predictedTime = trainModel().predict(self.model,self.createNpArray())
        return predictedTime


if __name__ == '__main__':
```

Figure 23 Continuous Learning

```python
def partialTraining(self,model,X_new,Y_new):
    onlineLearning = model.partial_fit(X_new,Y_new)
    return onlineLearning
```

Figure 24 Partial Training method

3.5 Testing.

3.5.1 Testing the initial model

Figure 24 shows the code used to evaluate my model's performance. The Root Mean Squared Error (RMSE) is a measure of prediction error, and the output score suggests that, on average, the model predicts a variance of 18.86 minutes. Considering the dataset ranged from 1771 minutes to 0.01 minutes, this result was mediocre for an initial model. I discuss in Section 3.6 how I refined the model.

The R-squared is a score between 0 and 1 to show how well the model fits the data points. The score of 0.8 indicates that the model is relatively accurate at predicting the time required to run a given distance, given age group and gender.

```python
        self.model.fit(X_train, y_train)
        # Generate predictions
        y_pred = self.model.predict(X_test)

        # Calculate metrics
        mse = mean_squared_error(y_test, y_pred)
        rmse = np.sqrt(mse)
        r2 = r2_score(y_test, y_pred)

        print(X_test)
        print(f"Model Performance:")
        print(f"- RMSE: {rmse:.2f}")
        print(f"- R² Score: {r2:.2f}")
```

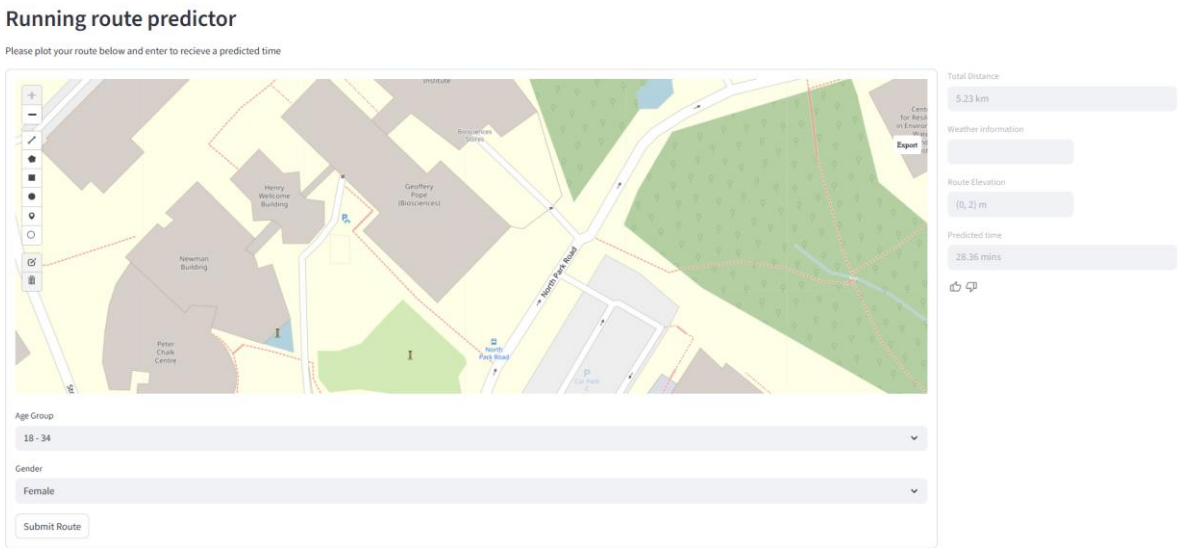
```
Model Performance:
- RMSE: 18.86
- R² Score: 0.80
```

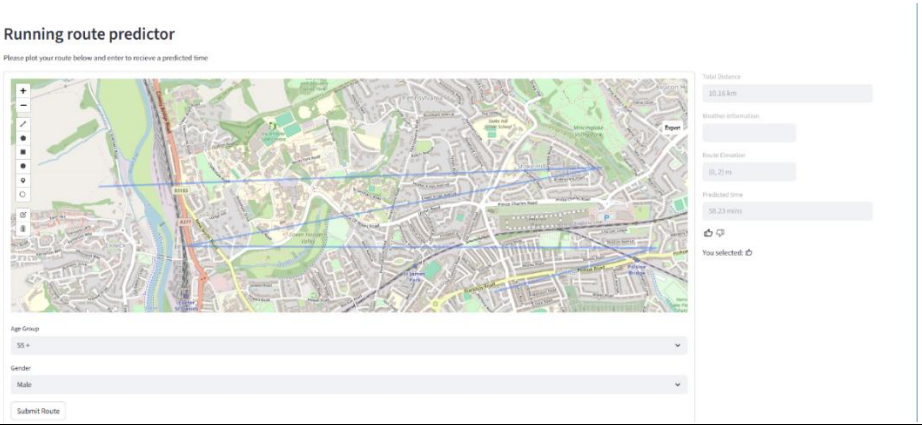
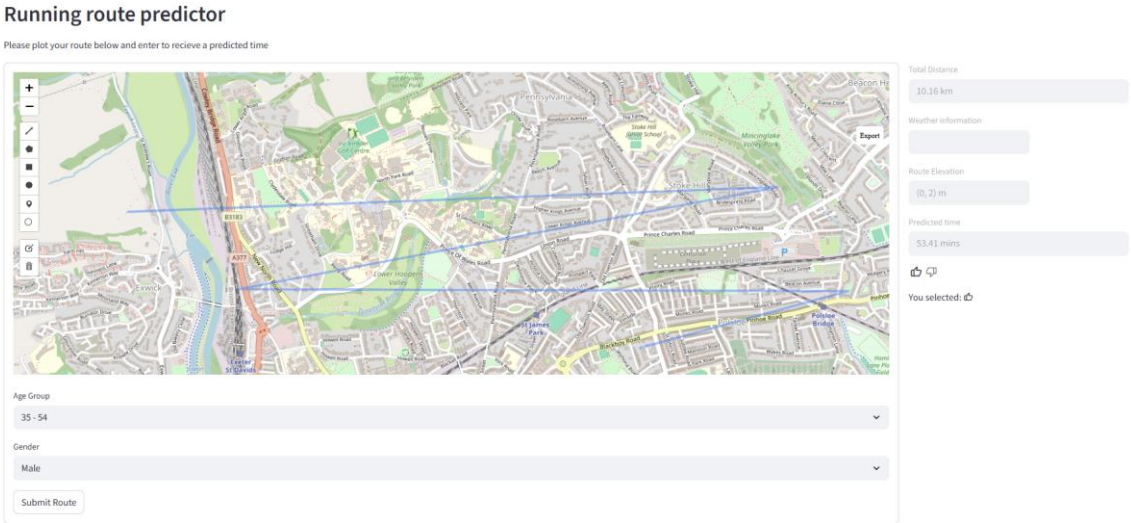
Figure 25 Metrics to evaluate performance

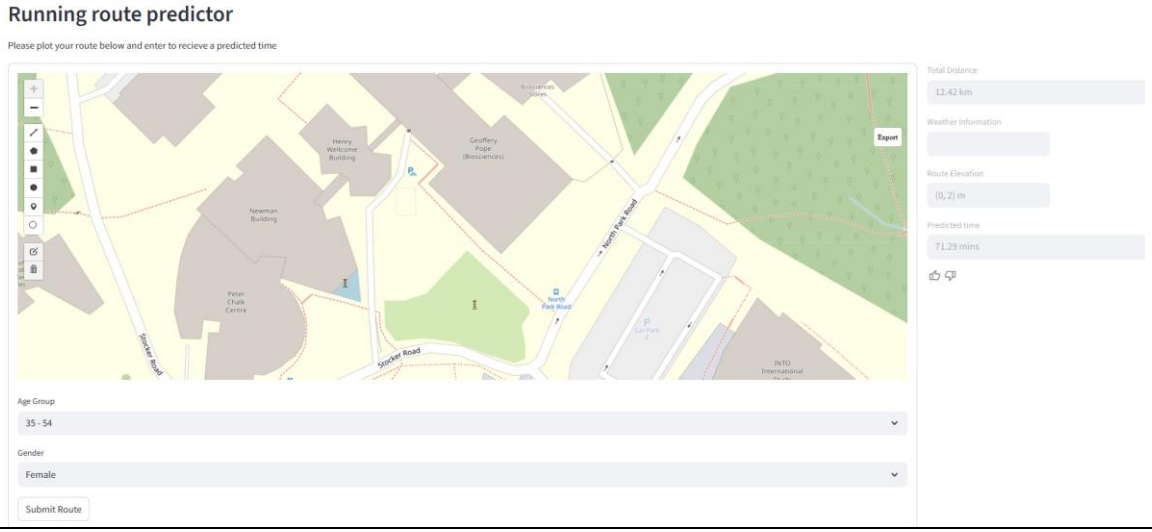
3.5.2 Testing the model with real-world data

Table 5 shows how my application compares the times to real-world statistics.

Table 5 Test Cases evidence

Test case	**Real-world data**	**Application prediction**
18–34-year-old woman to run 5km	23:07- 35:00 minutes (Running level, n.d.)	28.36 mins 

A 55+ year old male to run 10km.	48 mins - 2 hours (age dependent) (Running Level, n.d.)	58 minutes 
35-54-year-old woman running 10km.	50 – 70 minutes (Running Level, n.d.)	53 minutes 

35-54-year-old woman to run 12.42km	Distance is too specific for an average time, but the model proves it takes longer than 10km.	71.25 minutes 

3.6 Refining the model

Adding shuffle to the train_test_split

To prevent my model from learning patterns from the data's order, the data was shuffled before training. This didn't improve the model as the variance increased by 0.2.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.22, random_state=42,shuffle=True)
```

```
[83974 rows x 3 columns]
Model Performance:
- RMSE: 18.88
- R² Score: 0.80
None
```

Amendments to test size and random state.

Increasing the test size by 0.2 improved the model's reliability, with accuracy increasing by 0.1. The random state controls how many times the data is split per iteration, and decreasing it from 42 to 20 also reduced the RMSE score.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.22, random_state=20,shuffle=True)
```

```
[83974 rows x 3 columns]
Model Performance:
- RMSE: 17.96
- R² Score: 0.81
```

Changes to hyperparameters

Setting the learning rate to 'adaptive' made the time required to train the model variable during training. This is beneficial because, as the model trained, the learning rate decreased, preventing large differences in errors during fine-tuning that would occur during the final epochs.

```
def __init__(self):
    self.df=databaseUtils().convertToDf()
    self.model = SGDRegressor(max_iter=2000, alpha=0.0001, learning_rate='adaptive', random_state=42, shuffle=True, early_stopping= True)

def labelData(self):
```

Secondly, increasing the maximum iteration (max_iter) from 1000 to 2000 improved accuracy by giving the model more time to learn the patterns for predicting time. To prevent overfitting when changing max_iter, I set early_stopping = True, which ended training if scores weren't improving.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.22, random_state=20,shuffle=True)
```

This was the final model used for the application because of its relatively high accuracy, and, given the range of distances in the dataset, 17.67 minutes was the lowest variance achievable.

```
[83974 rows x 3 columns]
Model Performance:
- RMSE: 17.67
- R² Score: 0.82
```

4. Application code

4.1 UI

As discussed in section two, I chose Streamlit Folium as the front-end library. The UI is defined from line 68 onwards, with the map and text boxes arranged in columns. Lines 42 to 66 are the functions to call the classes that calculate distance, elevation, and predicted time.

The machine learning code is shown in Section 3, this section focuses on the delivery of the proof of concept with regards to the UI and business logic. . This section focuses on the delivery of the proof of concept with regard. This section focuses on the delivery of the proof of concept with regard to the UI and business logic

```
42  def set_total_distance(total_distance,age_group,gender):
43      st.session_state["route_data"] = total_distance
44      if gender=='Male':
45          gender_label = 'M'
46      else:
47          gender_label = 'F'
48      st.session_state["gender"]=gender
49      st.session_state['age_group']=age_group
50      if st.session_state["route_data"]['all_drawings'] == None:
51          st.write('Please plot a route before submitting')
52      else:
53          distanceElevation = calculateDistanceClass(total_distance['all_drawings']).calculateDistanceEvaluate()
54          print(f"{distanceElevation[0]:.2f} km")
55          global total_distance_Value
56          total_distance_Value = distanceElevation[0]
57          st.session_state["total_distance"] = f"{distanceElevation[0]:.2f} km"
58
59          if str(distanceElevation[1]) =='None' or str(distanceElevation[1]) == 0:
60              elevation = 0
61          else:
62              elevation = (round(int(distanceElevation[1])), 2)
63          st.session_state["elevation"] = str(elevation) + ' m'
64          predictedTime = machineLearningUtils(distanceElevation[0],age_group,gender_label).predictModel()
65          st.session_state["predicted_time_input"] = f"{predictedTime[0]:.2f} mins"
66          #st.session_state["weather"] = weatherClass().runEvent()
67
68  with col1:
69      with st.form("my_form"):
70          output = st_folium(m, width=2000 , height = 500) #OUTPUTTING THE MAP
71          age_group = st.selectbox('Age Group', ['18 - 34','35 - 54','55 +'])
72          gender = st.selectbox('Gender', ['Female','Male'])
73          st.form_submit_button('Submit Route' , key='Submit', on_click=set_total_distance, args=[output,str(age_group),gender])
74  with col2:
75      total_distance= st.text_input('Total Distance', key="total_distance" , disabled=True, label_visibility="visible", width="stretch")
76      weather= st.text_input('Weather information', key="weather" , disabled=True, label_visibility="visible", width=200)
77      elevation= st.text_input('Route Elevation', key="elevation" , disabled=True, label_visibility="visible", width=200)
78      predicted_time_input =st.text_input('Predicted time', key="predicted_time_input" , disabled=True, label_visibility="visible", width="stretch")
79      if st.session_state.predicted_time_input:
80          sentiment_mapping = [":material/thumb_down:", ":material/thumb_up:"]
81          selected = st.feedback("thumbs")
82          if selected is not None:
83                  st.markdown(f"You selected: {sentiment_mapping[selected]}")
84                  import re
85                  matches = re.findall(r'-?\d*\.?\d+', total_distance)
86                  res = [float(x) if '.' in x else int(x) for x in matches]      You, 1 second ago • Uncommitted changes
87                  if gender=='Male':
88                      gender_label = 'M'
89                  else:
90                      gender_label = 'F'
91                  st.session_state["gender"]=gender
92                  machineLearningUtils(res[0],st.session_state['age_group'],gender_label ).addToDB()
```

Figure 26 UI code

4.2 Calculating the distance, elevation and weather

Image 27 shows the functions to calculate the distance and elevation of my routes. The calculateDistance method requires a dataframe of longitudes and latitudes for each segment of the route and calls a method from Geopy to calculate the distance. An API call to openElevation determines the elevation for each set of coordinates. The ‘WeatherClass’ uses Python weather libraries to retrieve weather data for a particular location asynchronously.

```python
import requests

class calculateDistanceClass():

    def __init__(self,coordinatesList):
        self.coordinates = coordinatesList

    def calculateDistance(self,geometryList):
            df=pd.DataFrame(data=geometryList,columns=['LONGITUDE', 'LATITUDE'])
            dist=[0]
            for i in range(1,len(df)):
                dist.append(
                    geopy.distance.geodesic(
                        (df.LATITUDE.iloc[i], df.LONGITUDE.iloc[i]),
                        (df.LATITUDE.iloc[i - 1], df.LONGITUDE.iloc[i - 1])
                    ).km
                )
            df['distance']=dist
            totalGeometryDistance=sum(dist)

            return totalGeometryDistance

    def calculateDistanceEvaluate(self):
        """
        Main entry point to calculating the distance and elevation
        If there is onlly one set of lines plotted on the map, there will only be one set
        of co-ordinates
        Else, we must iterate through each segment of our route, find each distance/elevation
        separately and then add them all together

        """
        if (len(self.coordinates) ==1):
            totalGeoDistance =  self.calculateDistance(self.coordinates[0]['geometry']['coordinates'])
            totalElevationMultiple = self.calculateElevation(self.coordinates[0]['geometry']['coordinates'])
        else:
            totalGeoDistance = 0
            totalElevationMultiple = 0
            for i in range(0,len(self.coordinates)):
                totalGeoDistance = totalGeoDistance +  self.calculateDistance(self.coordinates[i]['geometry']['coordinates'])
                totalElevationMultiple  = totalElevationMultiple + self.calculateElevation(self.coordinates[i]['geometry']['coordinates'])

        return totalGeoDistance, totalElevationMultiple

    def calculateElevation(self, geometryList):
        elevationAnswer =0
        print(elevationAnswer)
        for i in range(0,len(geometryList) -1):
            url = 'https://api.open-elevation.com/api/v1/lookup'
            params = {'locations': f"{geometryList[1]},{geometryList[0]}"}
            response = requests.get(url, params=params)
            if response.status_code == 200:
                elevationAnswer = elevationAnswer+ response.json()['results'][0]['elevation']
            else:
                return 0
```

```python
import asyncio
import os
import datetime

class weatherClass():

    async def main(self) -> None:

        async with python_weather.Client (unit=python_weather.METRIC) as client:


            # Fetch a weather forecast from a city.
            weather = await client.get('Exeter')

            # Fetch the temperature for today.
            return weather.temperature

    # Fetch weather forecast for upcoming days.
    def runEvent(self):
        if os.name == 'nt':
            asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())

        temperature =asyncio.run(self.main())
        answer =  str(temperature) + ' degrees'

        return answer
```

Figure 27 Elevation, distance and weather calculation

5. Retrospective

Table 6 summarises the retrospective throughout the software cycle.

Table 6 Retrospective

Phase	What went well	What could have gone better	Learnings
Requirements and Ideation	Limited scope creep by selecting one machine learning use-case that aligned with my interests, ensuring requirements were specific.	Spent excessive time exploring datasets that didn't fit my use case, sometimes beginning development prematurely.	Learning how architectural characteristics and costs differ between traditional applications and machine learning products helped to adapt non-functional requirements to a business domain.
Design	Deciding on an architectural style was straightforward due to a clear set of non-functional requirements.	Visualising the designs was complex due to the distinction between model training and the application itself.	How to produce a basic sequence diagram to present a process architectural view.
Machine learning	Training the model was simpler than expected, producing predictions that accurately reflected real-world data.	The model rounded results to 10 decimal places, which was accurate, but it looked poor on the UI. The actual predicting of values is slow.	Completed a 9-hour live course on supervised learning as part of a 'fit4future' programme that my work offered. Successfully applied my learnings to the assignment.
Development and deployment	Developing and deploying on Streamlit Cloud, and testing the machine learning model and the UI, was a smooth experience with the excellent online documentation.	I initially used the Google Maps API, but calculating distances for more than two coordinates was complex due to the JSON format and treating coordinates as 'waypoints'. Switching to 'geopy', a Python library for geospatial tasks, eliminating the additional API.	Whilst a Kanban approach is flexible for software development, it was less motivating because there is no set deadline.

What would I do differently next time?

The overarching theme of what I found difficult was spending a lot of time working on solutions that didn’t succeed. In my next project, if I'm using a new tool or technology, the best approach would be to test it in practice before developing with it so I can determine more quickly what I can disregard. From a software methodology perspective, having SPIKE stories to experiment with a new technology would help me factor this into my own capacity and limit the pressure to force a redundant solution